

- **KẾ HOẠCH ĐÀO TẠO UNITY PUZZLE GAME (8 TUẦN)**
- **Đối tượng:** Intern đã hoàn thành “Fresher Training Program”
- **Định hướng sản phẩm:** Puzzle Games (Logic, Sorting, Packing, Match-3, Screw/Pin Puzzle) hướng tới tiêu chuẩn publish của **Voodoo**.
- **Engine & Công cụ:** Unity (LTS), C#, Git, Discord.
- **Thời lượng:** 8 tuần (chia làm 3 giai đoạn: Nền tảng → Gameplay Core → Polish & Production).

1. LỘ TRÌNH CHI TIẾT THEO TUẦN (WEEKLY MILESTONES)

GIẢI ĐOẠN 1: CHUYỂN DỊCH TƯ DUY TỪ WEB SANG UNITY & C# NÂNG CAO (TUẦN 1 - 2)

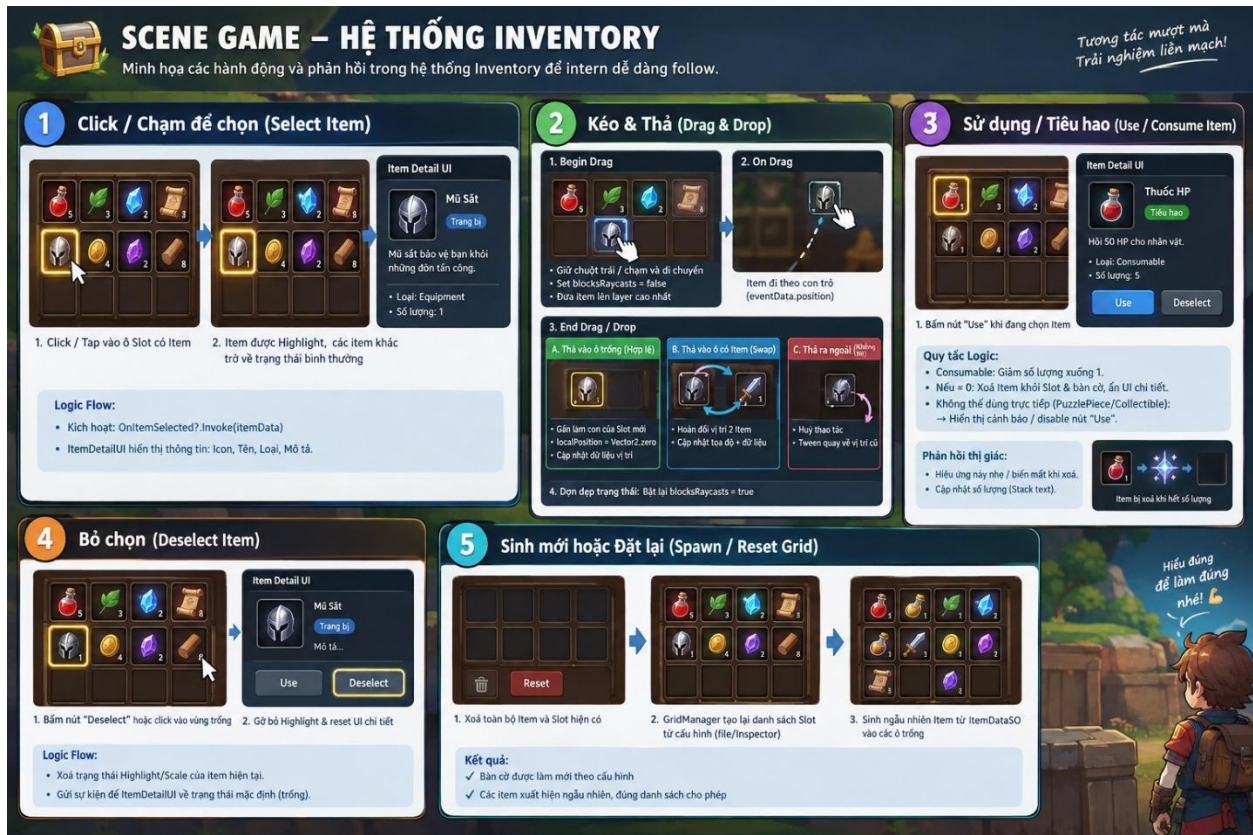
Mục tiêu: Đưa tư duy OOP, Matrix, Logic từ PIXIJS/TS sang Unity Engine & C# component-based.

- **Tuần 1: Unity Engine Basics, C# Architecture & Input System**
 - Làm quen Unity UI, Prefabs, ScriptableObjects, Game Loop (Update, FixedUpdate, LateUpdate).
 - Chuyển đổi tư duy: Từ Web DOM/Pixi Container sang Unity Hierarchy, Transform và Canvas (UGUI).
 - Viết code C# chuẩn convention: Clean Code, Event/Action (Observer Pattern), Singleton/Service Locator cơ bản.
 - Xử lý Input đa nền tảng (Touch/Mouse/Raycast 2D & 3D).
 - **Deliverable Tuần 1:** Make a **Simple Inventory System in Unity** (Kéo, thả, xem thông tin, sử dụng, xóa)

2. YÊU CẦU CHI TIẾT: TUẦN 1 - SIMPLE INVENTORY SYSTEM SCENE (DEADLINE: THỨ 6 – 25/09)

- **Chủ đề:** Unity Engine Basics, C# Architecture & Input System.

- **Mục tiêu chính:** Xây dựng một Scene giao diện túi đồ (Inventory System) hoàn chỉnh bằng UGUI, áp dụng ScriptableObject để quản lý dữ liệu, tách biệt Data – View – Controller, và xử lý mượt mà các tương tác kéo/thả, xem, dùng và xóa vật phẩm.
- **Nền tảng mục tiêu:** Mobile portrait responsive (Canvas Scaler chuẩn: 1080 × 1920).



1. PHÂN RÃ CÔNG VIỆC BÁM SÁT DELIVERABLE (TASK BREAKDOWN)

Task 1: Cấu trúc Dữ liệu Vật phẩm (Data Layer - ScriptableObject)

- Tạo class ItemDataSO kế thừa ScriptableObject gồm các trường:
 - id (string), itemName (string), icon (Sprite), type (Enum: Consumable, Equipment, Material), maxStack (int), description (string).
- Tạo menu [CreateAssetMenu] và cấu hình sẵn ít nhất **5 Item mẫu** khác nhau để test.
- Chuyển đổi tư duy: Dữ liệu được lưu độc lập dưới dạng asset tĩnh, không gắn cứng (hardcode) vào Prefab hay code hiển thị.

Task 2: Dựng Bàn cờ / Lưới Ô chứa (Grid & Slots)

- Dựng giao diện bằng UGUI Canvas:

- Sử dụng GridLayoutGroup để dựng lưới ô chứa (ví dụ: lưới 4 × 4 gồm 16 slots).
- Thiết lập Canvas Scaler ở chế độ *Scale With Screen Size*.
- Tạo Prefab SlotUI và ItemView:
 - SlotUI: Nắm giữ vị trí ô, lắng nghe sự kiện khi có Item thả vào.
 - ItemView: Hiển thị ảnh icon, số lượng (TextMeshPro), có component CanvasGroup để quản lý Raycast khi kéo thả.

Task 3: Tương tác Kéo & Thả (Drag & Drop Interface)

- Ứng dụng hệ thống EventSystem của Unity qua các interface: IBeginDragHandler, IDragHandler, IEndDragHandler, IDropHandler:
 - **Kéo (Drag):** Khi nhấn item, tạm thời tắt blocksRaycasts và đưa item lên layer cao nhất (transform.SetAsLastSibling()) hoặc chuyển tạm sang layer trung gian) để icon không bị che khuất.
 - **Thả hợp lệ (Drop):** Thả vào ô Slot trống → Item chuyển sang Slot mới; thả vào ô đã có Item → Hoán đổi vị trí (Swap) giữa 2 Item.
 - **Thả không hợp lệ (Invalid Drop):** Thả ra ngoài lưới → Tự động tween hoặc reset về lại Slot ban đầu.

Task 4: Xem Thông tin Vật phẩm (Inspect / Detail View)

- Bắt sự kiện Click / Tap vào ô Item:
 - Hiển thị trạng thái Highlight (viền sáng hoặc scale nhẹ) để báo hiệu đang được chọn.
 - Bảng thông tin chi tiết (ItemDetailUI) ở góc dưới màn hình cập nhật: Icon, Tên, Loại vật phẩm, Số lượng và Mô tả.
- Áp dụng **Observer Pattern (C# Action/Event)**: ItemView phát sự kiện OnItemSelected?.Invoke(itemData), ItemDetailUI lắng nghe và tự động đổ dữ liệu, không tham chiếu trực tiếp (loose coupling).

Task 5: Sử dụng & Xóa Vật phẩm (Use & Delete Logic)

- Trên bảng ItemDetailUI, tích hợp 2 nút bấm thao tác:
 - **Nút Use (Sử dụng):**
 - Kiểm tra loại item: Nếu là Consumable → Giảm stack đi 1.

- Khi stack về 0 → Tự động dọn dẹp và xóa Item khỏi ô chứa, đồng thời reset Detail Panel về trạng thái rỗng.
- Nếu không phải vật phẩm dùng được → Disable nút hoặc thông báo không hợp lệ.
- **Nút Delete / Remove (Xóa):**
 - Xóa bỏ ngay lập tức Item được chọn khỏi hệ thống Inventory và giải phóng ô Slot.
 - Ẩn bảng thông tin chi tiết.

2. YÊU CẦU KIẾN TRÚC & CODE CONVENTION

- **Kiến trúc phân tầng:** Tách biệt rõ 3 phần: **Model** (ItemDataSO, dữ liệu slot), **View** (ItemView, SlotUI, ItemDetailUI), **Controller/Manager** (InventoryManager, InputHandler).
- **Code Style:** Đặt tên hàm PascalCase, biến private/local camelCase có tiền tố `_`, tuyệt đối tránh viết code spaghetti dồn toàn bộ vào một file UI duy nhất.
- **Quản lý sự kiện:** Giao tiếp giữa các màn hình và hệ thống quản lý inventory phải dùng System.Action hoặc UnityEvent.

3. TIÊU CHÍ NGHIỆM THU (DELIVERABLE CHO BÁO CÁO THỨ 6)

1. **Gameplay Scene hoàn chỉnh:**
 - Grid hiển thị đầy đủ các slots và tự động nạp các item từ dữ liệu mẫu.
 - Kéo - thả (Drag & Drop) mượt mà giữa các ô, tự hoàn vị trí khi thả trượt.
 - Click chọn item hiển thị chuẩn xác thông tin lên Detail Panel.
 - Nút "Use" giảm đúng số lượng; nút "Delete/Deselect" dọn dẹp sạch sẽ dữ liệu trên giao diện.
2. **Không lỗi Console:** Không phát sinh lỗi NullReferenceException hoặc kẹt trạng thái kéo thả khi spam click chuột/chạm tay.